JAVASCRIPT · TEMA 5 · POO Y GESTIÓN DE OBJETOS

Programación orientada a objetos en JavaScript

Una guía para pasar de las propiedades y métodos a las clases, la herencia y los prototipos. Con ejemplos y actividades del tema.

objetos · propiedades · this · clases · herencia · recorridos · actividades

01 ¿Qué es la POO?

La programación orientada a objetos organiza el código en **objetos**. Un objeto reúne **datos** (propiedades) y **acciones** (métodos) relacionados entre sí.

Objeto: viaje → **origen, días, precio** + **mostrar()**

Ejemplo mental

Un viaje tiene un destino y un precio; también puede saber mostrarse o calcular un descuento.

JavaScript utiliza **prototipos** para compartir comportamiento entre objetos. La sintaxis `class` (incorporada en ES2015) facilita escribir ese código, pero continúa usando prototipos por debajo.

Si vienes de Java

En Java sueles definir primero una clase. En JavaScript también puedes crear directamente un objeto con `{ ... }` y acceder a sus propiedades sin declarar una clase.

02 Gestión de objetos

Trabajar con objetos supone saber **crearlos**, leer y cambiar sus propiedades, llamar a sus métodos, recorrerlos y, cuando convenga, compartir comportamiento mediante herencia.

Propiedad

Un dato asociado al objeto, por ejemplo `viaje.precio`.

Método

Una función asociada al objeto, por ejemplo `viaje.mostrar()`.

La idea clave

Piensa qué datos pertenecen a cada entidad y qué operaciones tiene sentido que realice sobre ellos.

03 Propiedades y métodos

La notación con **punto** se usa cuando conoces el nombre de la propiedad. Los **corchetes** aceptan un texto o una variable con ese nombre.

JavaScript · acceso a propiedades

```
const vector = [4, 2, 7, 9];
console.log(vector.length);       // 4
console.log(vector["length"]);    // 4

const nombre = "length";
console.log(vector[nombre]);      // 4
```

Los métodos también admiten las dos notaciones. Para ejecutarlos, añade `()`.

JavaScript · llamadas a métodos

```
console.log("Hola");
console["log"]("Hola");
```

Error típico

`viaje.mostrar` obtiene la función; `viaje.mostrar()` la ejecuta.

04 El operador `instanceof`

Comprueba si el prototipo de un constructor aparece en la cadena de prototipos de un objeto. Suele utilizarse para preguntar si un objeto es instancia de una clase o constructor.

JavaScript

```
const vector = [4, 2, 7];
console.log(vector instanceof Array); // true

const conjunto = new Set();
console.log(conjunto instanceof Map); // false
```

Lectura rápida

`vector instanceof Array` pregunta: «¿está `Array.prototype` en la cadena de prototipos de `vector`?».

05 Crear objetos con `Object`

`Object` es un constructor general. La mayoría de objetos ordinarios heredan de `Object.prototype`. Podemos crear uno e incorporar propiedades después.

JavaScript · objeto construido paso a paso

```
const notas = new Object();
notas.valores = [7.5, 3.2, 9.6];
notas.cantidad = notas.valores.length;

notas.verMedia = function () {
  const suma = this.valores.reduce((a, b) => a + b, 0);
  return suma / this.cantidad;
};

console.log(notas.verMedia().toFixed(2)); // "6.77"
```

Qué ocurre aquí

El objeto guarda los valores y un método para calcular su media. `toFixed(2)` devuelve un texto con dos decimales.

05 El literal de objeto

La forma más habitual de crear un objeto directamente es el **literal de objeto**, escrito entre llaves. Puede contener tanto datos como métodos.

JavaScript · literal de objeto

```
let viaje = {
  origen: "Granada",
  destino: "El Cairo",
  dias: 8,
  precio: 750,
  mostrar: function () {
    console.log(`${this.origen} / ${this.destino}`);
    console.log(`durante ${this.dias} días: ${this.precio} €`);
  }
};

viaje.mostrar();
```

Literal de objeto ≠ JSON

Se parecen visualmente, pero JSON es un formato de datos: sus claves van entre comillas y no admite funciones. El ejemplo anterior es código JavaScript.

06 ¿Qué significa `this`?

En una llamada como `viaje.mostrar()`, `this` dentro de `mostrar` señala al objeto que aparece antes del punto: `viaje`.

JavaScript · dos variables, un mismo objeto

```
const oferta = viaje;
viaje = null;

oferta.mostrar(); // Funciona: this es oferta
```

Una referencia no es una copia

`oferta` sigue apuntando al objeto. Asignar `null` a la variable `viaje` no destruye ese objeto mientras otra referencia lo conserve.

07 Clases y constructores

Cuando necesitamos muchos objetos con la misma estructura, una `class` permite definir cómo se inicializan y qué métodos comparten. `new` crea una instancia y ejecuta `constructor`.

JavaScript · varios viajes con una clase

```
class Viaje {
  constructor(origen, destino, dias, precio) {
    this.origen = origen;
    this.destino = destino;
    this.dias = dias;
    this.precio = precio;
  }

  mostrar() {
    console.log(`${this.origen} / ${this.destino}`);
    console.log(`durante ${this.dias} días: ${this.precio} €`);
  }
}

const miViaje = new Viaje("Barcelona", "Ibiza", 2, 112);
miViaje.mostrar();
```

new Viaje(...) → **constructor(...)** → **miViaje** → **mostrar()**

Relación con Java

La escritura se parece a una clase de Java. En JavaScript, los métodos declarados así se comparten mediante `Viaje.prototype`.

Recuerda

Dentro de una clase, `this.origen` guarda una propiedad de cada instancia; `origen` es solo el parámetro recibido por el constructor.

08 Herencia y polimorfismo

La **herencia** permite especializar una clase: `Profesor` y `Alumno` comparten los datos y métodos de `Miembro`. Con `extends` se establece la relación y con `super(...)` se llama al constructor de la clase padre.

Miembro ↘ **Profesor** · **Alumno**

JavaScript · clase base

```
class Miembro {
  constructor(nombre, alta, estado) {
    this.nombre = nombre;
    this.alta = alta;
    this.estado = estado;
  }

  cobrar() {
    console.log(`El Miembro ${this.nombre} ha cobrado`);
  }
}
```

JavaScript · primera clase hija

```
class Profesor extends Miembro {
  constructor(nombre, alta, estado, alumnos) {
    super(nombre, alta, estado);
    this.alumnos = alumnos;
  }

  cobrar() {
    console.log(`El Profesor ${this.nombre} ha cobrado`);
  }
}
```

Qué aporta `super(...)`

Inicializa `nombre`, `alta` y `estado` usando el constructor de `Miembro`. Después, `Profesor` añade `alumnos`.

08 Herencia y polimorfismo · continuación

Una clase hija puede **sobrescribir** un método heredado. Esto permite invocar `cobrar()` en distintos objetos y obtener comportamientos diferentes: **polimorfismo**.

JavaScript · segunda clase hija

```
class Alumno extends Miembro {
  constructor(nombre, alta, estado, asignaturas) {
    super(nombre, alta, estado);
    this.asignaturas = asignaturas;
  }

  cobrar() {
    console.log(`El Alumno ${this.nombre} ha recibido su beca`);
  }
}
```

JavaScript · mismas llamadas, distintos resultados

```
const prof = new Profesor("Ana Ruiz", "01/10/2022", "vigente", 30);
const alum = new Alumno("Carlos Pérez", "02/11/2022", "vigente", 8);

prof.cobrar(); // El Profesor Ana Ruiz ha cobrado
alum.cobrar(); // El Alumno Carlos Pérez ha recibido su beca
```

Comprueba la herencia

`prof.nombre` existe aunque `Profesor` no lo asigne directamente. `prof instanceof Miembro` devuelve `true`.

09 Recorrer propiedades

`for...in` recorre claves **enumerables** del objeto, incluidas las heredadas que sean enumerables. Si solo quieres nombres de propiedades propias, puedes usar `Object.getOwnPropertyNames()`.

JavaScript · dos formas de recorrer

```
for (const propiedad in miViaje) {
  console.log(propiedad);
}

for (const propiedad of Object.getOwnPropertyNames(miViaje)) {
  console.log(propiedad);
}
```

Herramienta	Qué devuelve
`for...in`	Propiedades enumerables, propias y heredadas.
`Object.keys(obj)`	Claves propias enumerables.
`Object.getOwnPropertyNames(obj)`	Nombres propios, incluso no enumerables.

Con las clases del ejemplo

Los métodos de `class` no son enumerables. Por eso `cobrar` no suele aparecer en `for...in` al recorrer una instancia.

10 Eliminar propiedades

`delete` elimina una propiedad **propia** configurable del objeto. Después, leerla devuelve `undefined` si no existe otra propiedad con el mismo nombre en la cadena de prototipos.

JavaScript

```
delete miViaje.precio;
console.log(miViaje.precio); // undefined
```

ACTIVIDADES PRÁCTICAS · 01

A1 Primeros objetos: usuario

Crea un objeto `usuario` con nombre, nombre de usuario, contraseña y un método `login()`. El método compara las credenciales recibidas con las guardadas en el objeto.

JavaScript · ejemplo del tema

```
const usuario = {
  nombre: "Antonio García",
  nombreUsuario: "agar007",
  contraseña: "agar007pass",

  login: function (nombreUsuario, contraseña) {
    if (nombreUsuario === this.nombreUsuario &&
        contraseña === this.contraseña) {
      console.log("Sesión iniciada con éxito");
    } else {
      console.log("Credenciales no válidas");
    }
  }
};

usuario.login("agar007", "agar007pass");
```

Prueba dos casos

Ejecuta `login()` con credenciales correctas e incorrectas. Localiza en qué parte se usa `this` para acceder al objeto.

En aplicaciones reales

Este ejemplo sirve para practicar objetos y métodos. Una contraseña real no debe guardarse en claro dentro del código JavaScript del navegador.

ACTIVIDADES PRÁCTICAS · 02

A2 Constructores: teléfono

Crea la clase `Telefono` con CPU, RAM, almacenamiento, ancho, alto y número de cámaras. Añade un método `toString()` que devuelva una descripción del teléfono.

JavaScript · ejemplo del tema

```
class Telefono {
  constructor(cpu, ram, almacenamiento, ancho, alto, numCamaras) {
    this.cpu = cpu;
    this.ram = ram;
    this.almacenamiento = almacenamiento;
    this.ancho = ancho;
    this.alto = alto;
    this.numCamaras = numCamaras;
  }

  toString() {
    return `${this.cpu} | ${this.ram} GB RAM | ` +
      `${this.almacenamiento} GB | ${this.ancho} × ${this.alto} cm | ` +
      `${this.numCamaras} cámaras`;
  }
}

const movil1 = new Telefono("Snapdragon 8 Gen2", 8, 128, 7.5, 15, 3);
const movil2 = new Telefono("Dimensity 9000", 12, 256, 7.2, 15.5, 4);

console.log(movil1.toString());
console.log(movil2.toString());
```

Comprueba el resultado

Las dos instancias comparten el método `toString()`, pero cada una conserva sus propios valores de CPU, RAM y dimensiones.

ACTIVIDADES PRÁCTICAS · 03

A3 Recorridos con herencia

Utiliza los objetos `prof` y `alum` del apartado 08. Recorre sus propiedades con `for...in` y con `Object.getOwnPropertyNames()`. Compara qué muestra cada uno.

JavaScript · punto de partida

```
for (const propiedad in prof) {
  console.log(propiedad, prof[propiedad]);
}

for (const propiedad of Object.getOwnPropertyNames(prof)) {
  console.log(propiedad, prof[propiedad]);
}
```

¿Por qué podrían salir iguales?

Los métodos declarados en `class` no son enumerables. Para observar la diferencia, añade temporalmente `Miembro.prototype.centro = "Arangoya"` y repite ambos recorridos.

✓ Chuleta rápida

Quiero...	Uso...
Crear un objeto directo	`const obj = { ... }`
Crear varias instancias	`class`, `constructor`, `new`
Acceder al objeto que llama	`this.propiedad`
Especializar una clase	`extends` y `super(...)`
Recorrer claves propias	`Object.keys(obj)`
Eliminar una propiedad	`delete obj.propiedad`